# Lecture 2. Finite Automata

Spring 2026, *Theory of Computation*, IIIS, Tsinghua

# Abstract Models

- Finite Automata
  - Only finite number of states
  - this lecture
- Pushdown Automata
  - Finite automata with an infinite stack
- Turing Machines
  - Finite automata with an infinite tape



# Why Abstract Models

- What is a good ***physical model*** of a "computer"
  - Computing with strands of DNA
  - Computing with neurons
  - Computing with circuits (etched in silicon wafers)
- Abstractions are easier to reason about than real computers
  - Much easier to learn math techniques in this context
- One model can represent ***infinitely many computers***
  - We can prove something about all such computers at once



# Human Languages

- English, Mandarin, Danish, Arabic, …
- Which sentence are ***true***, ***meaningful***, ***grammatical***?
  - *dlkdj &*lkjd 2398kldjkljd*
  - *was exam didn't I know when. retake?*
  - *Three is divisible by two.*
  - *Sipser's textbook has more than 100 pages.*



# ***Formal*** Languages

- An ***alphabet***, $\Sigma$, is a finite set of symbols, e.g.,
  - $\Sigma = \{a,\ldots,z\}$
  - $\Sigma = \{0,1\}$
  - $\Sigma = \{0,\ldots,9,+,-,\times,/,(,)\}$
  - $\Sigma = \{\circledast, \maltese, \cup, \partial\}$



# ***Formal*** Languages

- An ***alphabet***, $\Sigma$, is a finite set of symbols, e.g.,
  - $\Sigma = \{a,\ldots,z\}$
  - $\Sigma = \{0,1\}$
  - $\Sigma = \{0,\ldots,9,+,-,\times,/,(,)\}$
  - $\Sigma = \{\circledast, \maltese, \cup, \partial\}$
- A ***string*** is a sequence of zero or more symbols from $\Sigma$, e.g., $\maltese\maltese\maltese$ and $\circledast\partial$ are strings.
- The empty (zero length) string is called "$\varepsilon$"
- $\Sigma^* = \{\varepsilon\} \cup \Sigma \cup \Sigma^2 \cup \Sigma^3 \cup \Sigma^4 \ldots$ is the set of all finite length strings over $\Sigma$
- A ***language*** is a subset of $\Sigma^*$



# Example Formal Languages

- $\Sigma$ = {x, y, z, ∧, ∨, ¬, (, )}
- Languages:
  - $L_0$ = set of all ***syntactically correct*** Boolean formulas
  - $L_1$ = set of all Boolean ***tautologies*** *(always true)*

- Of course $L_1 \subseteq L_0$



# Example Formal Languages

- $\Sigma = \{x, y, z, \wedge, \vee, \neg, (, )\}$
- Languages:
  - $L_0$ = set of all ***syntactically correct*** Boolean formulas
  - $L_1$ = set of all Boolean ***tautologies***
- Examples:

  $x \wedge y$

  $x \vee \neg x$

  $(x \vee y \vee (\neg x \wedge \neg y))$

  $(x\ (\vee\ y\ \neg()$



# Example Formal Languages

- $\Sigma = \{x, y, z, \wedge, \vee, \neg, (, )\}$
- Languages:
  - $L_0$ = set of all ***syntactically correct*** boolean formulas
  - $L_1$ = set of all boolean ***tautologies***
- Examples:

| | |
|---|---|
| $x \wedge y$ | $\in L_0$ |
| $x \vee \neg x$ | $\in L_1 \subseteq L_0$ |
| $(x \vee y \vee (\neg x \wedge \neg y))$ | $\in L_1 \subseteq L_0$ |
| $(x\ (\vee\ y\ \neg()$ | $\notin L_0$ |



# What is a program/machine?

- Some thing M *(**what** this is depends on the model)*
  - That takes a string as input *(**how** depends on the model)*
  - Outputs "accept" or "reject" *(**how** depends on the model)*
  - L(M) *(the **language** of M)* is the set of all strings that makes M output "accept"

**Q.** Why just allow 1-bit outputs ("accept" vs. "reject")?
**A.** We're trying to *simplify our models as much as possible*. The goal is to reason about what computational models can/cannot do.
Multiple output: several such machines.



# Deterministic Finite Automata



# Example 1

- 25¢ toll booth



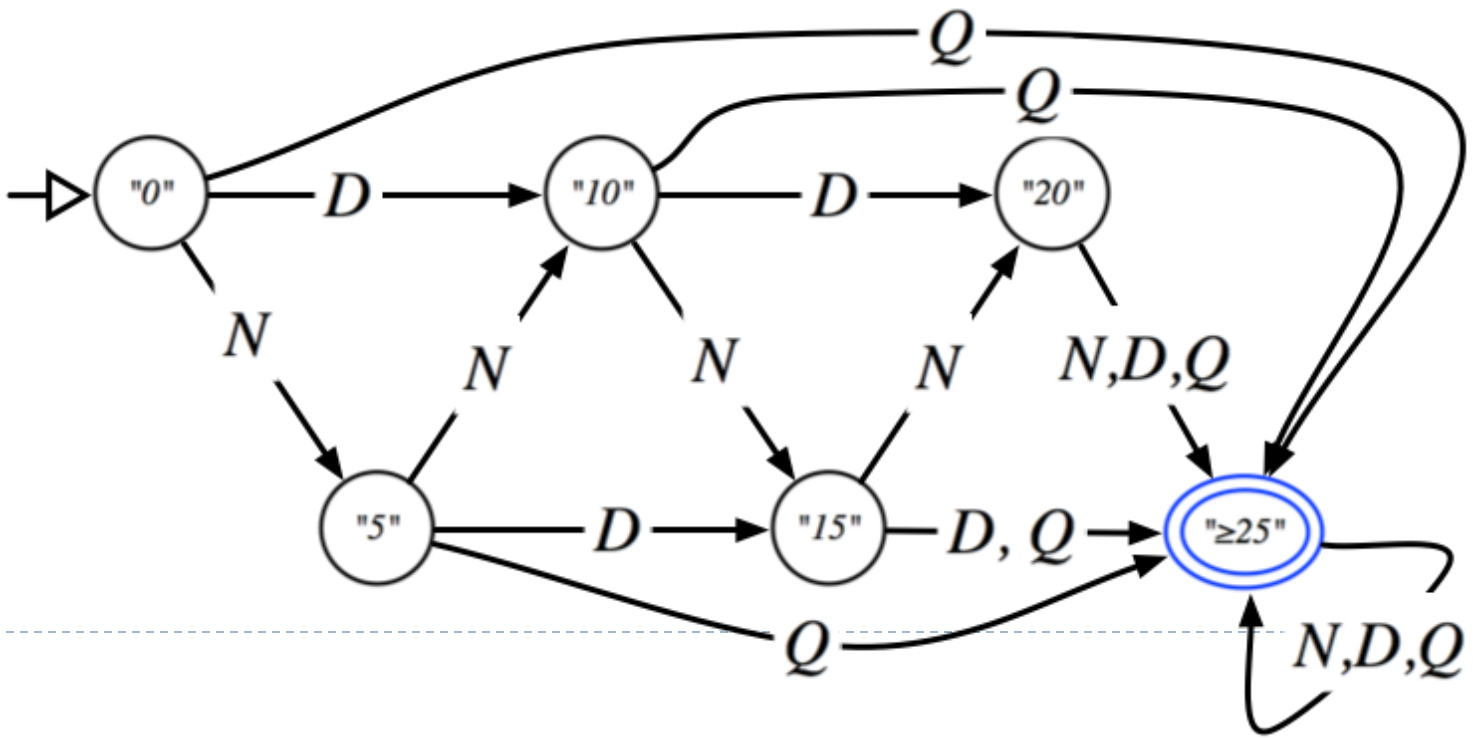

- Input is a sequence of coins N,D,Q
  (nickel 5¢, dime 10¢, quarter 25¢)
- States keep track of amount deposited
- A car can pass if sum of coins is ≥ 25¢





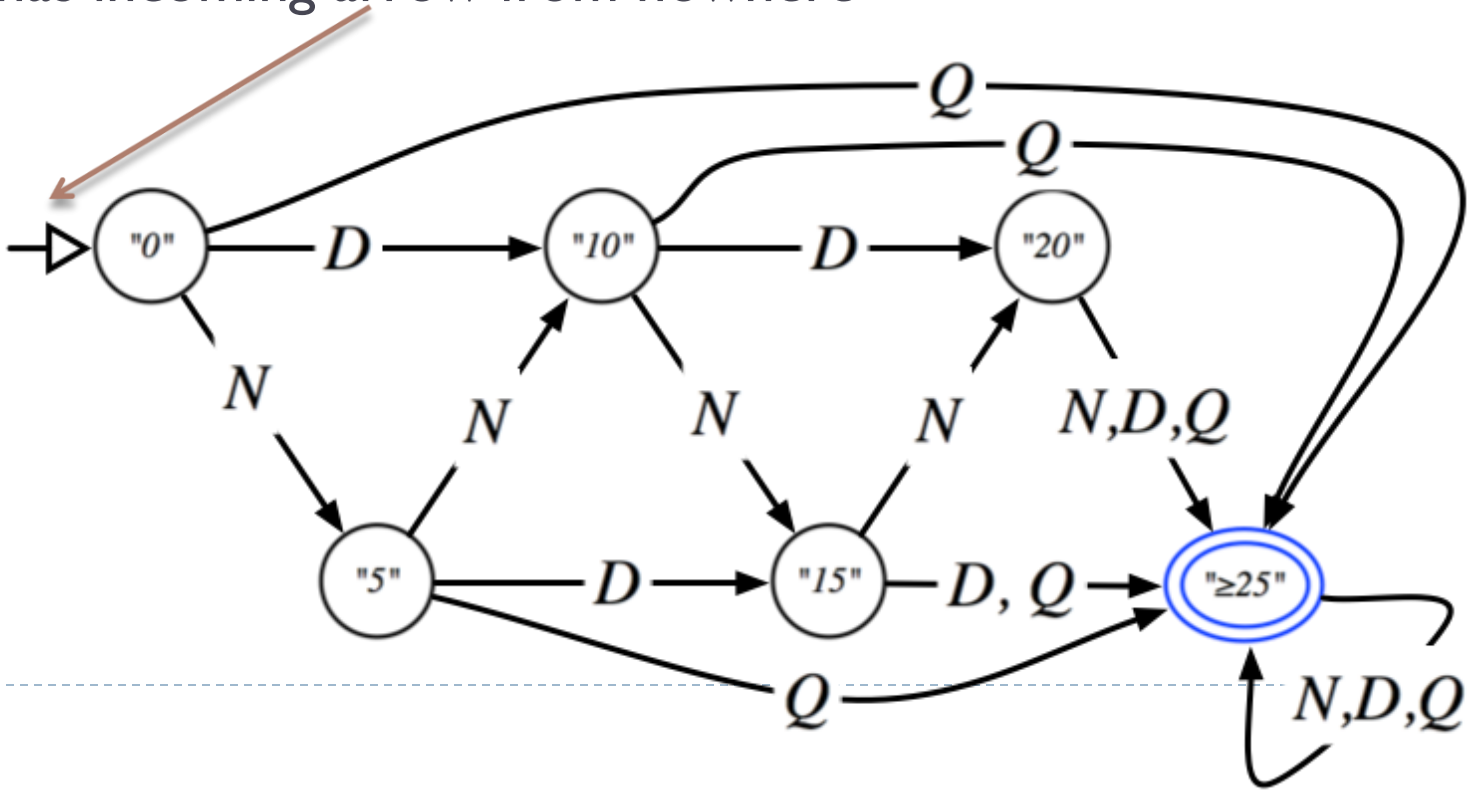

# Example 1

- State diagrams



# Example 1

- State diagrams
  - States = circles
  - Labeled arrows = state transitions
  - Accept states = double circles
  - Start state has incoming arrow from nowhere



# Finite Automata

- ***Fixed*** amount of memory (equivalently, fixed number of **states**)
- ***Start state***, zero or more ***accept states***
- Input is a sequence of symbols (over some alphabet)
- Reads symbols *one at a time*
- Transition function, $\delta$, describing changes of state



# Finite Automata

- ***Fixed*** amount of memory (equivalently, fixed number of **states**.)
  - The size of memory is constant
  - But we can still do a lot of things!

- ***Start state***, zero or more ***accept states***
- Input is a sequence of symbols (over some alphabet)
- Reads symbols *one at a time*
- Transition function, $\delta$, describing changes of state



# Finite Automata

"Automaton" is the singular form
"Automata" is the plural form

- ***Fixed*** amount of memory (equivalently, fixed number of **states**.)
- ***Start state***, zero or more ***accept states***
- Input is a sequence of symbols (over some alphabet)
- Reads symbols *one at a time*
- Transition function, $\delta$, describing changes of state
  - Unlike Turing machine, finite automaton always halts, and the running time is the same as the length of input.



# Accepting strings

- Our model:
  - The input is a ***finite string***
  - The string is either ***accepted*** or ***rejected*** depending on the current state when the input ends
- Syntax vs. Semantics
  - Syntax（语法学）: ***what the automaton is*** (states, transition function, etc.)
  - Semantics（语义学） ***what the automaton does*** (which strings the automaton accepts)



# Formal defn. of a DFA

- *"Deterministic Finite Automaton"*
- A 5-tuple $(Q,\Sigma,\delta,q,F)$
  - $Q$ is a ***finite*** set of ***states***
    - can give them informative names if you want
    - be comfortable with $Q = \{q_1,q_2,\ldots q_{|Q|}\}$
  - $\Sigma$ is a ***finite*** set of ***symbols*** (the alphabet)
  - $\delta : Q \times \Sigma \rightarrow Q$ is the ***transition function***
  - $q \in Q$ is the ***start state***
  - $F \subseteq Q$ is the set of ***accepting states***



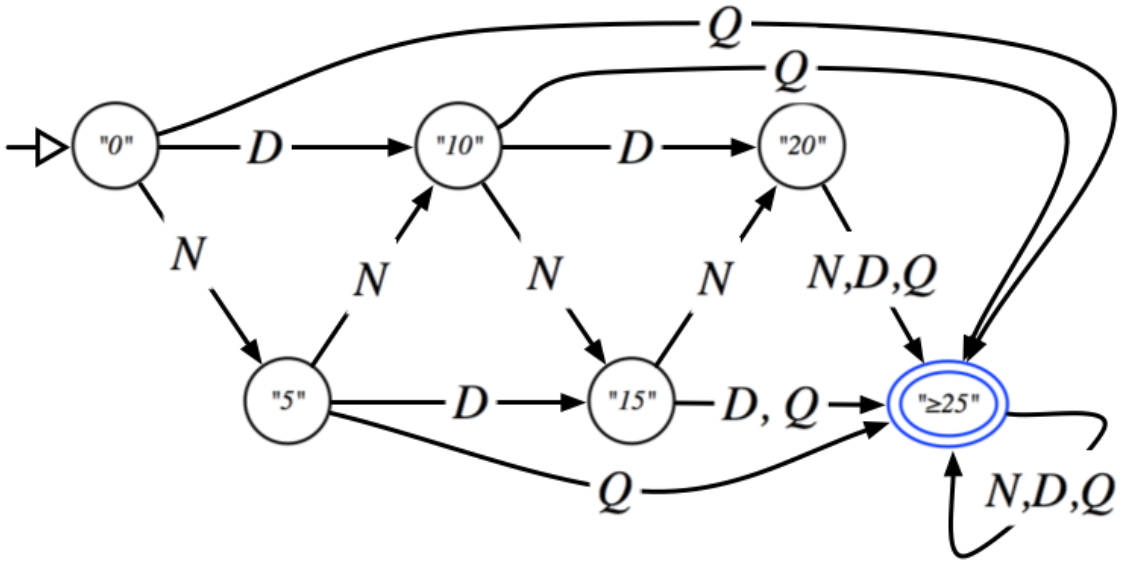
# Formal description of Example 1

- ({“0”,“5”,“10”,“15”,“20”,“≥25”},
  {N,D,Q},
  $\delta$,
  “0”,
  {“≥25”})

*states*
*alphabet*
*trans. funct.*
*start state*
*set of accepting states*

$\delta$:

| | N | D | Q |
|---|---|---|---|
| “0” | “5” | “10” | “≥25” |
| “5” | “10” | “15” | “≥25” |
| “10” | “15” | “20” | “≥25” |
| “15” | “20” | “≥25” | “≥25” |
| “20” | “≥25” | “≥25” | “≥25” |
| “≥25” | “≥25” | “≥25” | “≥25” |



# Syntax vs. Semantics

- Syntax: what the automaton *is* (5-tuples $(Q,\Sigma,\delta,q,F)$)
- Semantics:

  Give meaning to: **"Automaton M accepts string w"**
- Intuitive explanation: *"If you walk around the state diagram of M according to the sequence w, you end in an accept state."*



# Formal Defn. of "Acceptance"

- What does it mean for DFA M to accept w?
  - $M = (Q,\Sigma,\delta,q,F)$
  - $w = w_1 w_2 \ldots w_n$ each $w_i \in \Sigma$ for $1 \le i \le n$

- Write a logical formula for "M accepts w"
- $\exists (r_0,r_1,\ldots,r_n)$ where $r_i \in Q$ for $0 \le i \le n$
  and $r_0 = q$
  and $r_i = \delta(r_{i-1},w_i)$, for $1 \le i \le n$
  and $r_n \in F$



# Languages

- A ***language*** is a (possibly infinite) set of strings over some alphabet

- L(M) = {w | M accepts w}

  *"the language recognized by M"*

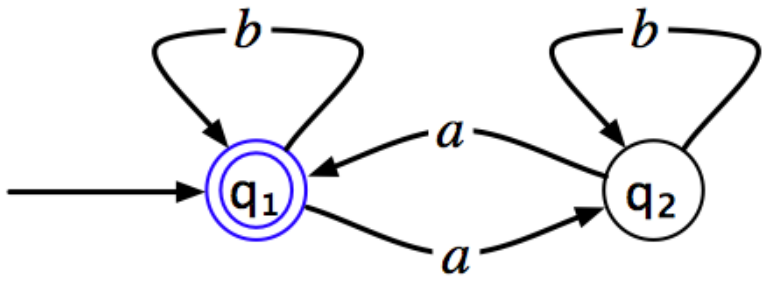
- A machine always recognizes one language!
  - If it accepts no strings, it recognizes the "empty language" Ø



# iClicker quiz

- $L(M_1)$ = {w | "_____" }

$M_1$:

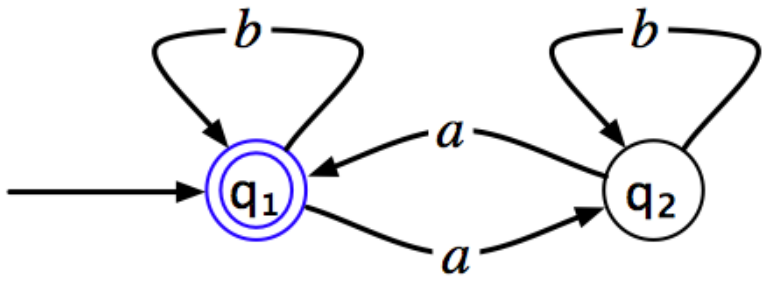





# iClicker quiz

- $L(M_1)$ = {w | "w has even number of 'a'" }

$M_1$:

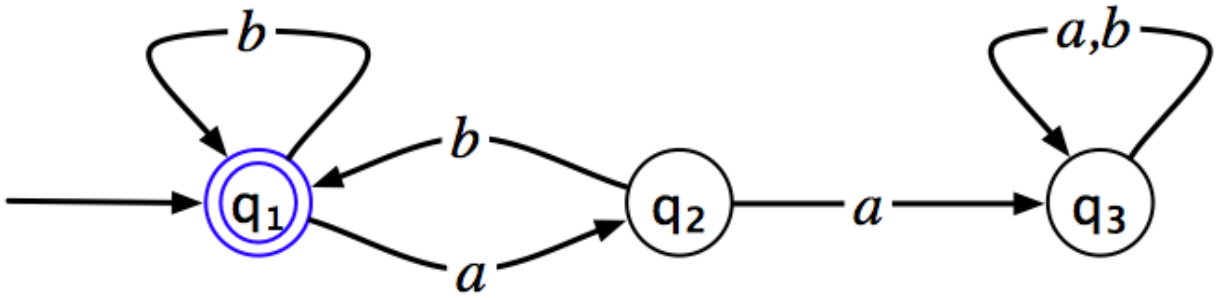





# iClicker quiz

- $L(M_2)$ = {w | "_____" }

$M_2$:

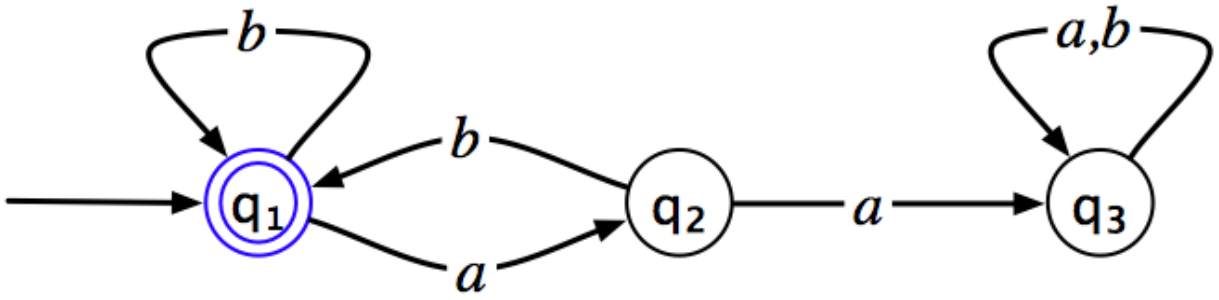





# iClicker quiz

- $L(M_2)$ = {w | "w doesn't contain '*aa*' and ends with a '*b*'" }
- or: "each '*a*' followed by a '*b*'

$M_2$:






# Design DFA

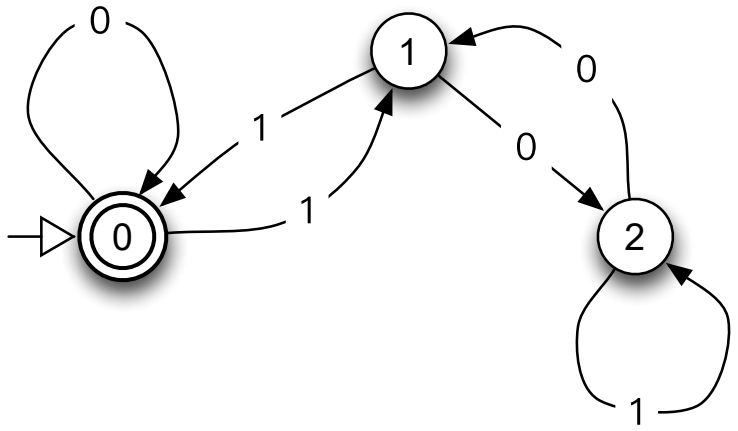
- Is there a DFA M such that:
  - $\Sigma=\{0,1\}$
  - L(M)={binary number that can be divided by 3}



# Design DFA

- Is there a DFA M such that:
  - $\Sigma=\{0,1\}$
  - L(M)={binary number that can be divided by 3}

  - state: x mod 3, transition: (2x+i) mod 3






# Design DFA

- Is there a DFA M such that:
  - $\Sigma = \{a,b,c\}$
  - L(M) = {first letter only appears once}



# Design DFA

- Is there a DFA M such that:
  - $\Sigma = \{a, b\}$
  - L(M) = {w | w does not contain ***abab***}



# Capabilities of DFA

- Identifying simple patterns
  - 1st letter = last letter
  - each *a* followed by *b*
- Arithmetic with a bounded counter
  - counting coins at a toll booth
  - ***modular arithmetic***



# Regular lauguage

- ***Definition.*** *Regular Languages*: The set of *all* languages recognized by *some* DFA

- What can we say about regular languages?
  - Closure properties
  - How to build new regular languages from old ones



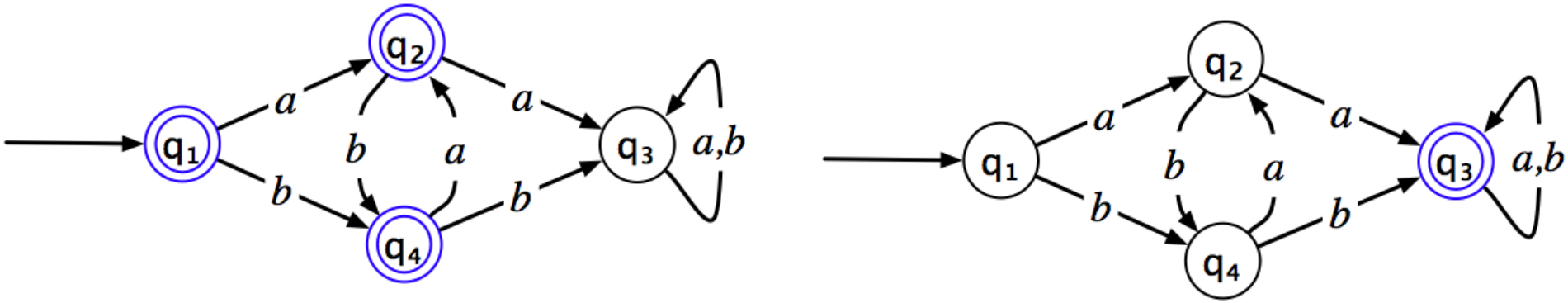
# Building new automata from old ones

- How can we find an DFA M':
  - $$L(M') = \overline{L(M)} = \{w \in \{a,b\}^* \mid w \notin L(M)\}$$



# Building new automata from old ones

- How can we find an DFA M':
  - $$L(M') = \overline{L(M)} = \{w \in \{a,b\}^* \mid w \notin L(M)\}$$




**Theorem.** The class of regular languages is closed under complementation

**Proof.** M' identical to M, except with final states Q\F



# Are Reg. langs. closed under intersection?

- $M_1, M_2$ two DFAs
- Is there an $M_3$ with $L(M_3) = L(M_1) \cap L(M_2)$ ??
  - *remember: languages are just sets of strings*
- e.g., $L(M_1) = \{w \mid w$ has at least 2 ***b***s$\}$

  $L(M_2) = \{w \mid w$ has an even # of ***a***s$\}$

  $L(M_3) = \{w \mid w$ has at least 2 ***b***s

  *and* an even # of ***a***s$\}$
- Find a ***systematic*** way to build $M_3$!



# Are Reg. langs. closed under intersection?

- $M_1, M_2$ two DFAs
- Is there an $M_3$ with $L(M_3) = L(M_1) \cap L(M_2)$ ??

- A great method
  - **Run $A_1$** on the input string (note if it accepts)
  - **Rewind the input** **<= (not supported by DFA)**
  - **Run $A_2$** on the input string
  - **Accept if both $A_1$ and $A_2$ accept**



# Building new automata from old ones

- $M_3$: Imagine running $M_1$ and $M_2$ ***simultaneously***
- Formally:
  - $M_1 = (Q_1, \Sigma, \delta_1, s_1, F_1)$
  - $M_2 = (Q_2, \Sigma, \delta_2, s_2, F_2)$
- How should $M_3$ be ***formally defined***?



# Building new automata from old ones

- $M_3$: Imagine running $M_1$ and $M_2$ ***simultaneously***
- Formally:
  - $M_1 = (Q_1, \Sigma, \delta_1, s_1, F_1)$
  - $M_2 = (Q_2, \Sigma, \delta_2, s_2, F_2)$
- How should $M_3$ be ***formally defined***?

$M_3 = (Q_1 \times Q_2, \Sigma, \delta_3, (s_1,s_2), F_1 \times F_2)$

where $\delta_3((r_1,r_2), a) = (\delta_1(r_1,a), \delta_2(r_2,a))$

for each $(r_1,r_2) \in Q_1 \times Q_2$ and $a \in \Sigma$

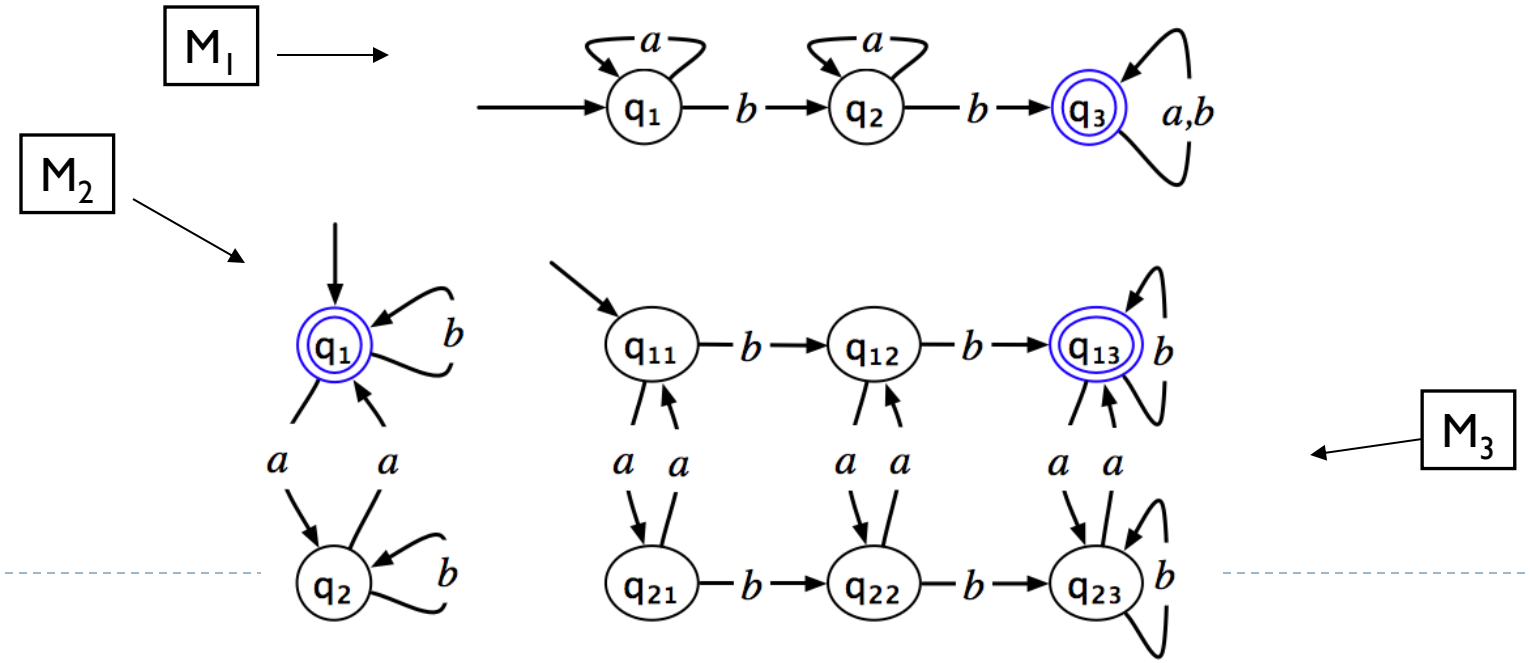


# Building new automata from old ones

- $M_1, M_2$ two DFAs
- Is there an $M_3$ with $L(M_3) = L(M_1) \cap L(M_2)$ ??
- Example: $L(M_1) = \{w \mid w \text{ has } \geq 2 \textbf{ b}\text{s}\}$

  $L(M_2) = \{w \mid w \text{ has an even \# of } \textbf{a}\text{s}\}$

  $L(M_3) = \{w \mid w \text{ has } \geq 2 \textbf{ b}\text{s} \quad \underline{\textit{and}} \quad \text{an even \# of } \textbf{a}\text{s}\}$



# Building new DFAs from old ones

- $M_1, M_2$ two DFAs
- Is there an $M_3$ with $L(M_3) = L(M_1) \cup L(M_2)$ ??



# Building new DFAs from old ones

- $M_1, M_2$ two DFAs
- Is there an $M_3$ with $L(M_3) = L(M_1) \cup L(M_2)$ ??
- DeMorgan's Law (for sets)

$$X \cup Y = \overline{(\bar{X} \cap \bar{Y})}$$



# Regular Operations on Languages

- A & B are two (possibly infinite) sets of strings
- **Union:** $A \cup B = \{x \mid x \in A \text{ or } x \in B\}$
- **Concatenation:**
  - $A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$
- **Star:**
  - $A^* = \{x_1 x_2 \ldots x_k \mid k \geq 0 \text{ and all } x_i \in A\}$
  - *note: the empty string $\varepsilon$ (k=0) is always in $A^*$*



# Regular Operations on Languages

- A & B are two (possibly infinite) sets of strings
- **Union:** $A \cup B = \{x \mid x \in A \text{ or } x \in B\}$ ✅
- **Concatenation:** ?
  - $A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$
- **Star:** ?
  - $A^* = \{x_1 x_2 \ldots x_k \mid k \geq 0 \text{ and all } x_i \in A\}$
  - *note: the empty string $\varepsilon$ ($k=0$) is always in $A^*$*



# Are DFAs Closed Under Concatenation? and Under * (Repeated Concatenation)?

- $M_1$ recognizes $L(M_1)$
- $M_2$ recognizes $L(M_2)$
- Is there an $M_3$ recognizing $L(M_1) \circ L(M_2)$ ??
- 1st try for $M_3$ (which doesn't work)
  - Simulate $M_1$ until it first enters an accepting state
  - Then simulate $M_2$ on rest of input
  - Accept if and only if $M_2$ accepts



# Why it's not that easy

- $L(M_1)$ = {w | w ends with ***abba***}
- $L(M_2)$ = {w | w does not contain ***abab***}
- Is ***abbabbababbabbabaab*** $\in L(M_1) \circ (M_2)$ ??



# Why it's not that easy

- $L(M_1)$ = {w | w ends with ***abba***}
- $L(M_2)$ = {w | w does not contain ***abab***}
- Is ***abbabbababbabbabaab*** $\in L(M_1) \circ (M_2)$ ??

***abba***bbababbabbabaab

***abbabba***bababbabaab

***abbabbababbabba***baab

How to decide when to switch from $M_1$ to $M_2$?



# Nondeterministic Finite Automata

# ***Nondeterministic*** Finite Automata (NFA)

- Like a DFA ***except***
  - Can change state *spontaneously*

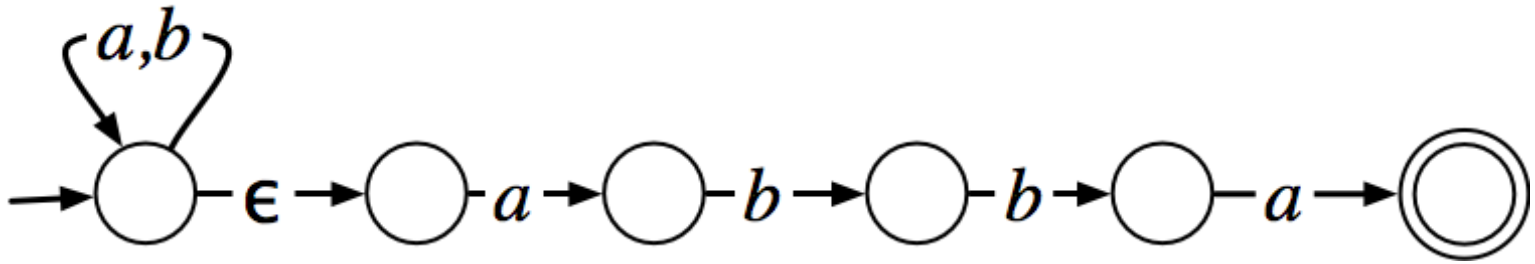
    i.e., read $\varepsilon$ (the empty string) and switch states
  - Can be 0, 1, or more "next" states given current state and next symbol (or $\varepsilon$)

- What does it mean for an NFA to accept a string?



# iClicker

- This NFA N:




(a) accepts ‘εabba’, rejects ‘abba’

(b) accepts ‘ababba’, rejects ‘abbba’

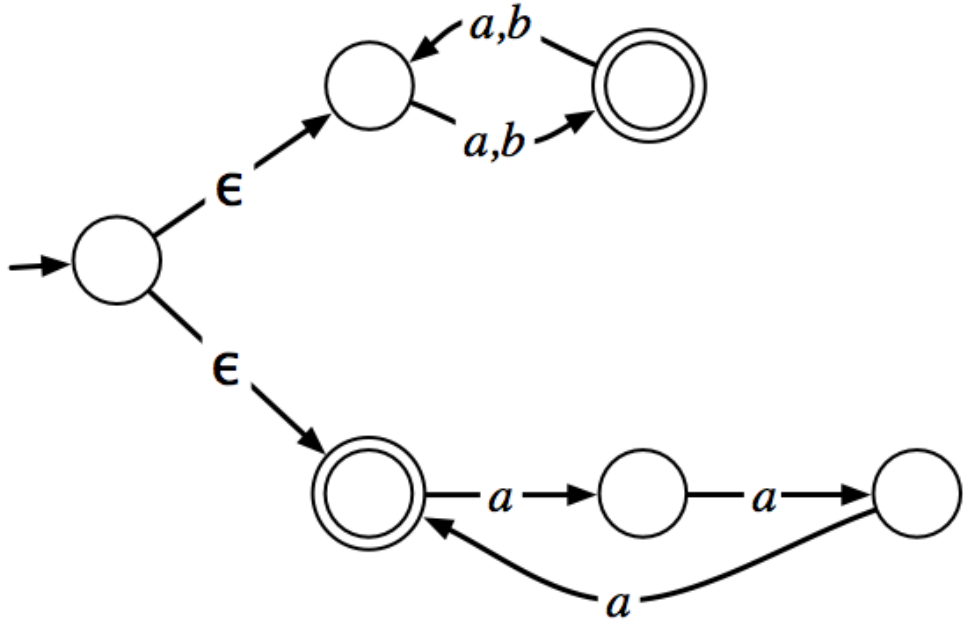
(c) accepts ‘abbab’, rejects ‘abab’

- L(N) = {w ∈ {***a***,***b***}$^*$ | w ends with ***abba***}



# iClicker

- L(N) = {w ∈ {***a***,***b***}* | "_____"}
- N:




(a) |w| is odd and w contains 3n ***a***s for some n≥0

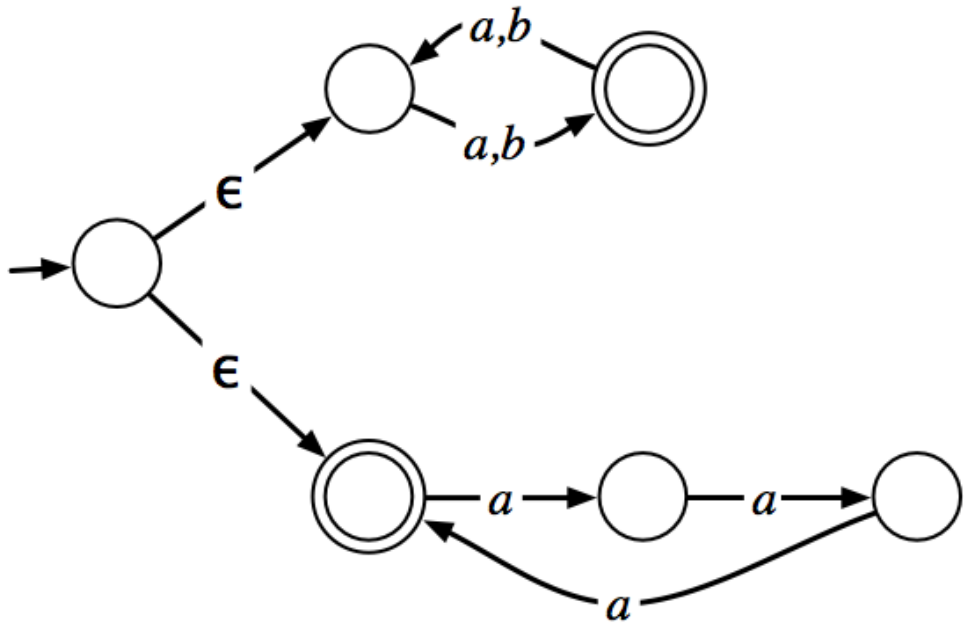
(b) |w| is even and w contains 3n ***a***s for some n≥0

(c) |w| is odd or w=$\boldsymbol{a}^{3n}$ for some n≥0



# iClicker

- L(N) = {w ∈ {***a***,***b***}* | "_____"}
- N:




(a) |w| is odd and w contains 3n ***a***s for some n≥0

(b) |w| is even and w contains 3n ***a***s for some n≥0

(c) |w| is odd or w=$a^{3n}$ for some n≥0 ✅

# *Formally* Defining an NFA

- Need to take into account '$\varepsilon$' labels
  - New alphabet $\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$
- … and multiple possible "next" states

- ***Definition:***

$2^A$ = the ***set of all subsets*** of A (the ***power set*** of A)

Example: $2^{\{x,y,z\}} = \{\emptyset, \{x\}, \{y\}, \{z\}, \{x,y\}, \{x,z\}, \{y,z\}, \{x,y,z\}\}$



# Formal Defn. of an NFA

- A 5-tuple $(Q,\Sigma,\delta,q,F)$
  - $Q$ is a ***finite*** set of ***states***
  - $\Sigma$ is a ***finite*** set of ***symbols*** (the alphabet)
  - $\delta : Q \times \Sigma_\varepsilon \rightarrow 2^Q$ is the ***transition function***
  - $q \in Q$ is the ***start state***
  - $F \subseteq Q$ is the set of ***accepting states***
- How is this different than a DFA?

$\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$
$\varepsilon$ = empty string

$2^Q$ = the ***power set*** of Q
= the set of all subsets of Q

# Formal Defn. of an NFA

- A 5-tuple $(Q,\Sigma,\delta,q,F)$
  - $Q$ is a ***finite*** set of ***states***
  - $\Sigma$ is a ***finite*** set of ***symbols*** (the alphabet)
  - $\delta : Q \times \Sigma_\varepsilon \rightarrow 2^Q$ is the ***transition function***
  - $q \in Q$ is the ***start state***
  - $F \subseteq Q$ is the set of ***accepting states***
- How is this different than a DFA?

was $\delta : Q \times \Sigma \rightarrow Q$

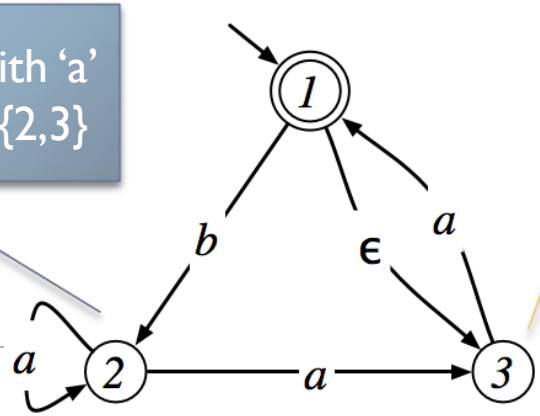


# Formal Defn. of an NFA

- A 5-tuple $(Q,\Sigma,\delta,q,F)$
  - $Q$ is a ***finite*** set of ***states***
  - $\Sigma$ is a ***finite*** set of ***symbols*** (the alphabet)
  - $\delta : Q \times \Sigma_\varepsilon \rightarrow 2^Q$ is the ***transition function***
  - $q \in Q$ is the ***start state***
  - $F \subseteq Q$ is the set of ***accepting states***

- How is this different than a DFA?

there are two arrows with 'a' from state 2, so $\delta(2,a)=\{2,3\}$




e.g. there is no arrow with 'b' from state 3, so $\delta(3,b)=\emptyset$



# Formal Defn. of "Acceptance"

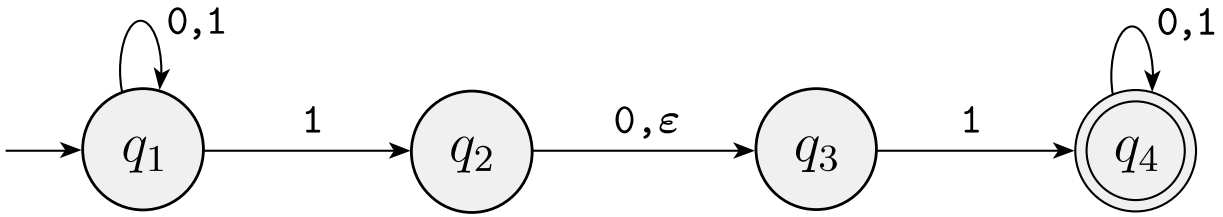
- NFA N accepts string w $\in \Sigma^*$ if:
  - $N = (Q,\Sigma,\delta,q,F)$
  - w can be written $x_1x_2\ldots x_m$ each $x_i \in \Sigma_\varepsilon$
    (e.g., 'abba' can be written 'a$\varepsilon$b$\varepsilon\varepsilon$ba')

- $\exists\ (r_0,r_1,\ldots,r_m)$ *a sequence of m+1 states*
  - $r_0 = q$
  - $r_m \in F$
  - $r_i \in \delta(r_{i-1},x_i)$ for $1 \le i \le m$

How does this differ from the DFA's defn. of acceptance?



# Example




- States Q={$q_1$, $q_2$, $q_3$, $q_4$}
- **Σ**={0,1}
- **δ**:

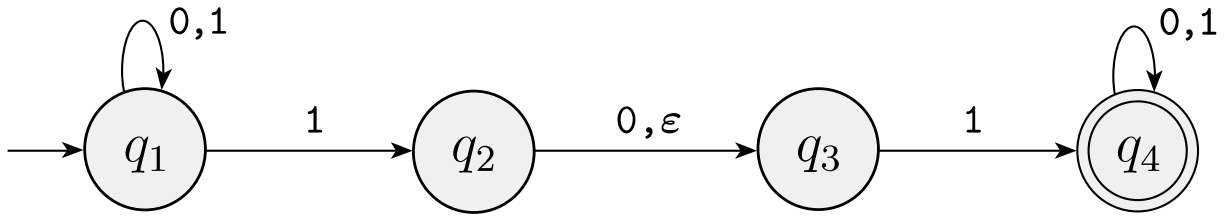
| | 0 | 1 | ε |
|---|---|---|---|
| $q_1$ | $\{q_1\}$ | $\{q_1, q_2\}$ | ∅ |
| $q_2$ | $\{q_3\}$ | ∅ | $\{q_3\}$ |
| $q_3$ | ∅ | $\{q_4\}$ | ∅ |
| $q_4$ | $\{q_4\}$ | $\{q_4\}$ | ∅, |

- $q_1$ is the start state
- F={$q_4$}

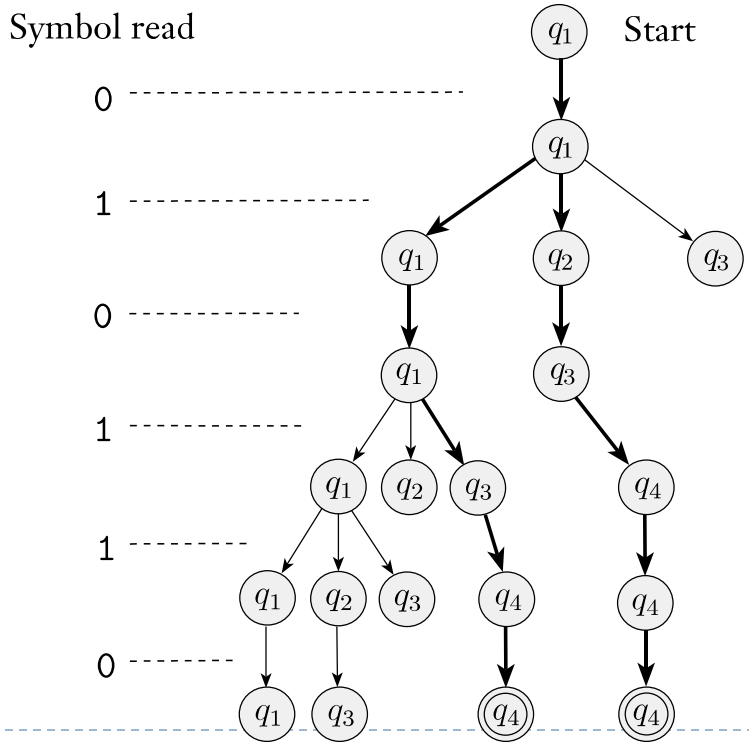


# Example

What does this NFA do?




The "computation tree" on input 010110



# Exercise

- Design an NFA N such that:
  - $\Sigma = \{a,b,c\}$
  - L(N)={last letter has appearance before}



# Exercise

- Design an NFA N such that:
  - $\Sigma=\{a,b,c\}$
  - L(N)={last letter has no appearance before}



# Equivalence of NFA and DFA

- DFA → NFA *trivial*
- NFA → DFA :
- ***Theorem.*** For any ***NFA*** N, there is a ***DFA*** M such that L(N) = L(M)



# NFA→DFA Transformation

- ***Theorem.*** For any ***NFA*** N, there is a ***DFA*** M such that L(N) = L(M)

- Main idea:
  - At all times, M keeps track of the ***set of states*** that N ***could*** be in

- Proof:
  - *First assume N has no ε-transitions*
  - Then we'll do the general case



# Without ε-transitions

- NFA: $N = (Q, \Sigma, \delta, q, F)$

- DFA: $M = (Q', \Sigma, \delta', q', F')$
  - $Q' = 2^Q$ $\{2^Q = \textit{power set of } Q\}$
  - $\delta'(R, a) = \cup_{r \in R} \delta(r, a)$ $R \in Q', a \in \Sigma$
  - $q' = \{q\}$
  - $F' = \{R \in Q' \mid R \cap F \neq \emptyset\}$



# Now with ε-transitions

- NFA: $N = (Q, \Sigma, \delta, q, F)$

- DFA: $M = (Q', \Sigma, \delta', q', F')$
  - $Q' = 2^Q$ $\{2^Q = \text{power set of } Q\}$
  - $\delta'(R, a) = \cup_{r \in R} \delta(r, a)$ $R \in Q', a \in \Sigma$
  - $q' = \{q\}$
  - $F' = \{R \in Q' \mid R \cap F \neq \emptyset\}$

How should $\delta'$,q' be fixed to handle ε-transitions?



# *Now with ε-transitions*

- NFA: N = (Q,**Σ**,$\delta$,q,F)
  - E(R) = {r ∈ Q | r is reachable from R using zero or more $\varepsilon$-transitions}

How should $\delta$',q' be fixed to handle $\varepsilon$-transitions?



# Now with ε-transitions

- NFA: $N = (Q, \Sigma, \delta, q, F)$
  - $E(R) = \{r \in Q \mid$ r is reachable from R using zero or more ε-transitions$\}$

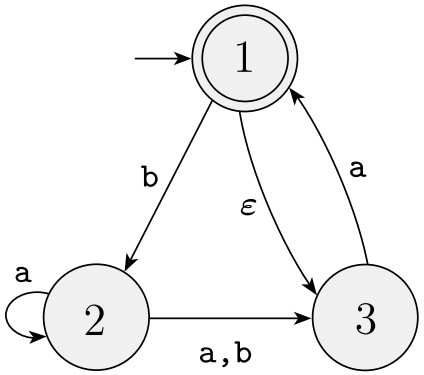
- DFA: $M = (Q', \Sigma, \delta', q', F')$
  - $Q' = 2^Q$
  - $\delta'(R, a) = \bigcup_{r \in R} E(\delta(r,a))$ $\quad R \in Q', a \in \Sigma$
  - $q' = E(\{q\})$
  - $F' = \{R \mid R \cap F \neq \emptyset\}$

How should δ',q' be fixed to handle ε-transitions?



# Example

- A 3-state NFA: How many states in the corresponding DFA?

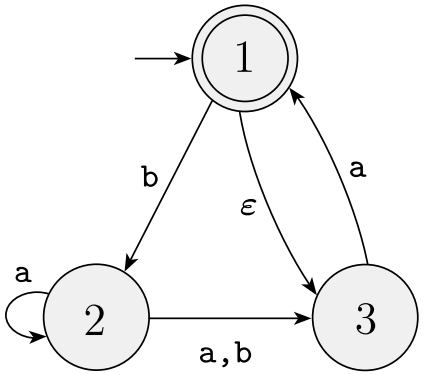


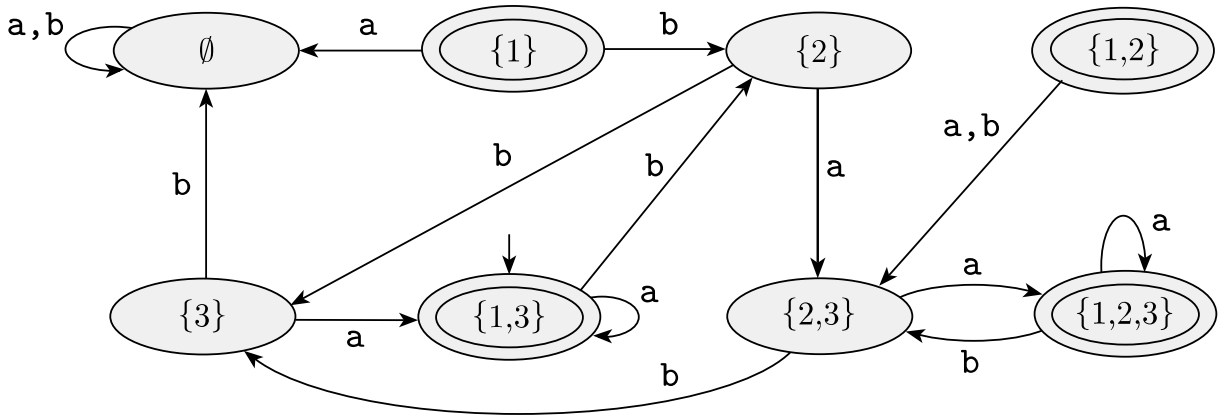

What's E({1}) ?



# Example

- {1,3} is the start state
- $2^3$ states in corresponding DFA

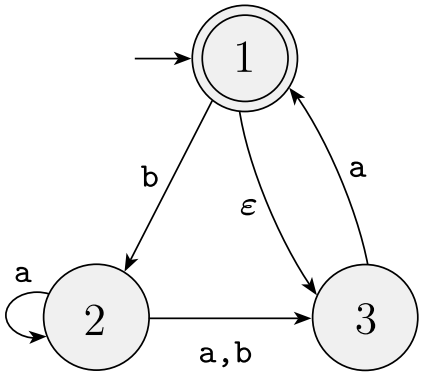


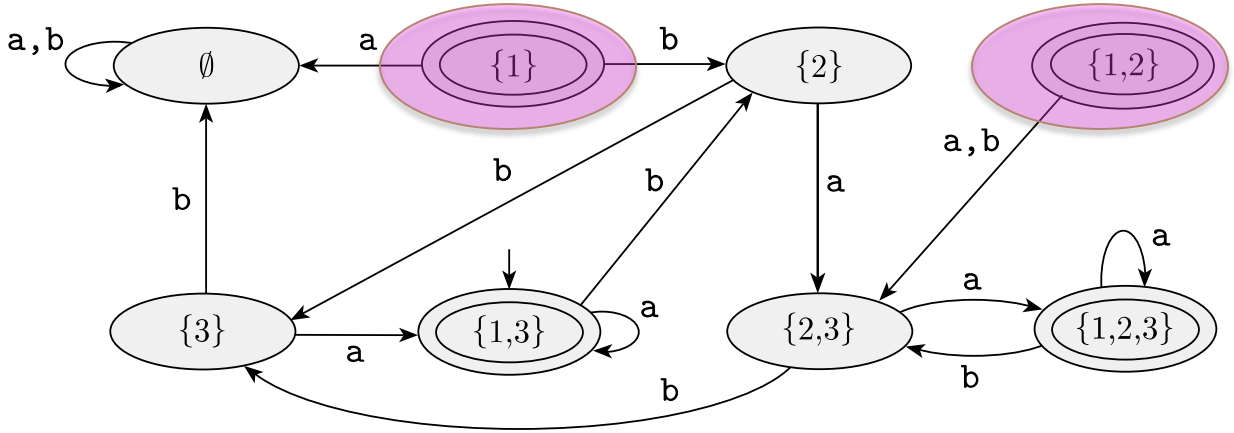






# Example

- We can simplify it by deleting unreachable states






- Number of states directly corresponds to ***memory requirements***
- A DFA (implicitly) needs $\log_2 |Q|$ bits of memory to remember which of $|Q|$ states it is in
- NFA with state set Q

  → DFA with state set $2^Q$

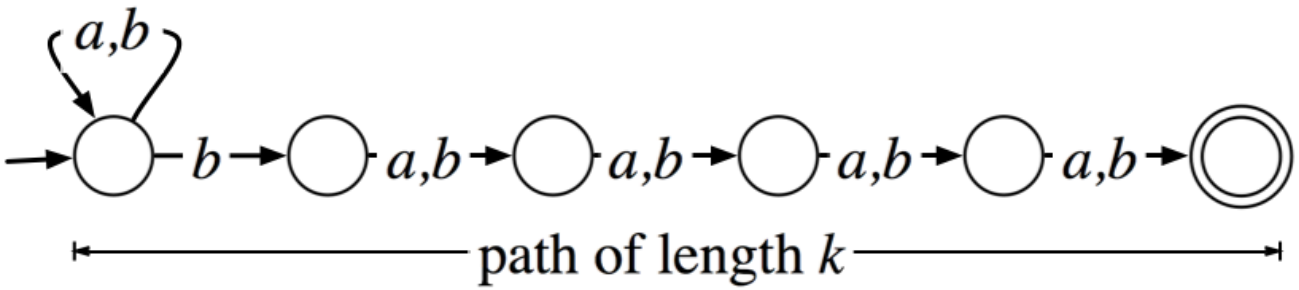
  → DFA needing $\log_2(2^{|Q|}) = |Q|$ bits of memory
- Is this exponential blowup necessary? Does non-determinism sometimes/always give an exponential improvement in the number of states?



# How much non-determinism helps

- $\Sigma = \{a,b\}$
- $L_k = \{w \in \{a,b\}^* \mid$ the $k^{th}$ symbol from the end is $b\}$

- Step 1. Find a (k+1)-state NFA recognizing $L_k$




example for k=5

- Step 2. Prove that any DFA with $< 2^k$ states does *not* recognize $L_k$



Step 2. Prove that any DFA with $< 2^k$ states does *not* recognize $L_k$

Proof: ***by contradiction***

1. Suppose there were such a DFA M with $|Q|=2^k-1$
2. Imagine running M on each input $w \in \{a,b\}^k$

$$|\{a,b\}^k| = 2^k > |Q|$$

3. By the pigeonhole principle,
   there exist $w,w' \in \{a,b\}^k$ and $r \in Q$ such that
   (i) $w \neq w'$
   (ii) After reading either w or w', M is in state r
4. Let j+1 be the first position where w and w' differ:

w = xxxxxxxxxxaxxxxxxxxxxxxx

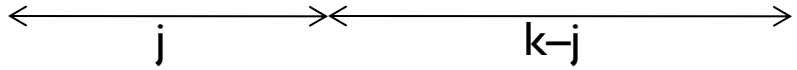
w' = xxxxxxxxxxbxxxxxxxxxxxxx



4. Let j+1 be the first position where w and w' differ:

$w = xxxxxxxxxx\boldsymbol{a}xxxxxxxxxxxxx$

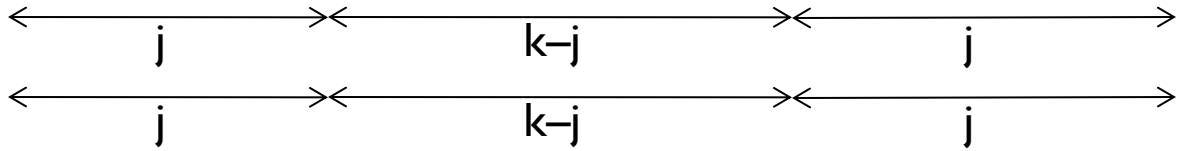
$w' = xxxxxxxxxx\boldsymbol{b}xxxxxxxxxxxxx$




5. Run M on $w\boldsymbol{a}^j$ and $w'\boldsymbol{a}^j$, on both inputs it ends in the same state, say r'

$w\boldsymbol{a}^j = xxxxxxxxxx\boldsymbol{a}xxxxxxxxxxxxx\boldsymbol{aaaaaaaaaaa}$

$w'\boldsymbol{a}^j = xxxxxxxxxx\boldsymbol{b}xxxxxxxxxxxxx\boldsymbol{aaaaaaaaaaa}$




6. Either $r' \in F$ or $r' \notin F$, and both cases lead to contradiction!



# Union, Concatenation, Star

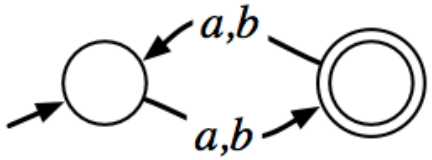
- Closure under regular operations:
- Given NFAs $N_1$ and $N_2$, find an $N_3$ such that
  - $L(N_3) = L(N_1) \cup L(N_2)$ union
  - $L(N_3) = L(N_1) \circ L(N_2)$ concatenation
  - $L(N_3) = L(N_1)^*$ star

- Exercise: take union, concat, star for
  - $L(N_1) = \{w \in \{\boldsymbol{a},\boldsymbol{b}\}^* \mid |w| \text{ is odd}\}$
  - $L(N_2) = \{w \in \{\boldsymbol{a},\boldsymbol{b}\}^* \mid w=\boldsymbol{a}^{3n} \text{ for some } n\geq 0\}$

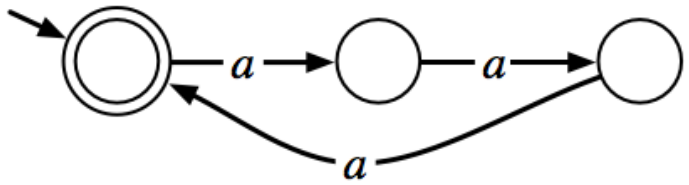


# NFAs Closed Under ***Union***, Concatenation

- For NFAs $N_1, N_2$, there is some NFA $N_3$
  - $L(N_3) = L(N_1) \cup L(N_2)$




$L(N_1) = \{w \mid |w| \text{ is odd.}\}$

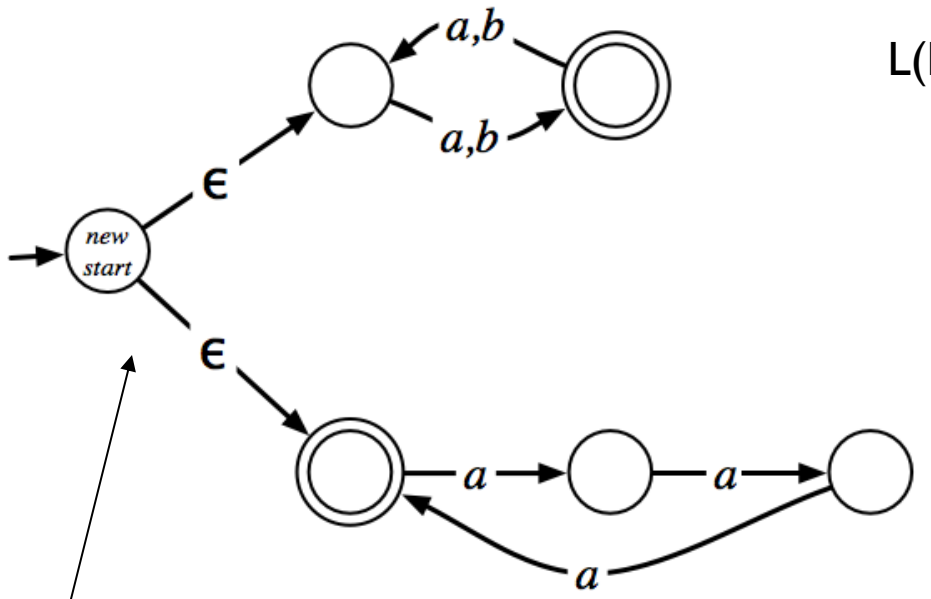



$L(N_2) = \{w \mid w = a^{3n} \text{ for some } n \geq 0\}$



# NFAs Closed Under ***Union***, Concatenation

- For NFAs $N_1, N_2$, there is some NFA $N_3$
  - $L(N_3) = L(N_1) \cup L(N_2)$




$L(N_1) = \{w \mid |w| \text{ is odd.}\}$

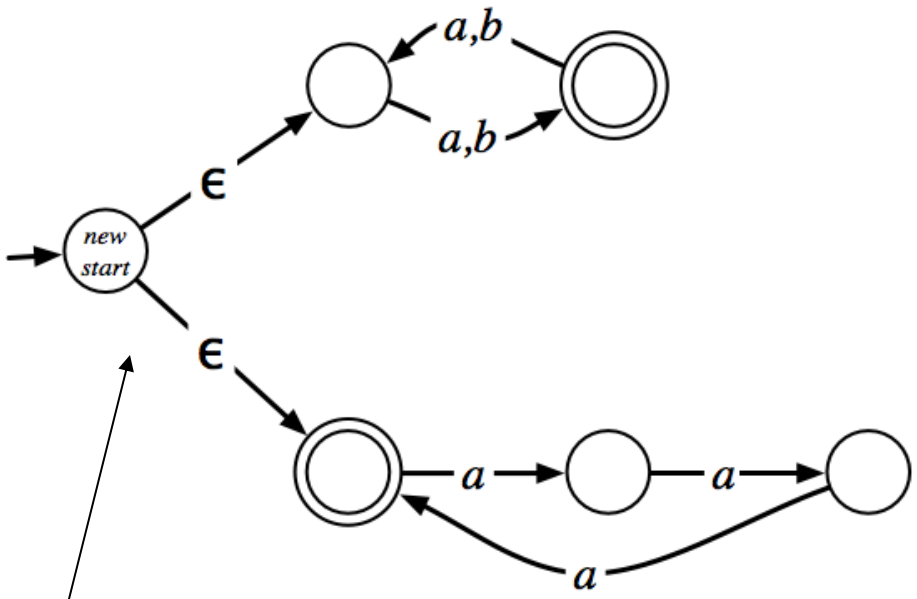
$L(N_2) = \{w \mid w = a^{3n} \text{ for some } n \geq 0\}$

Non-deterministically choose which NFA to run



# iClicker Quiz

- For any NFA $N_1$, there is an NFA $N_2$ such that $L(N_1) = L(N_2)$ and $N_2$ has exactly ***one*** accept state.



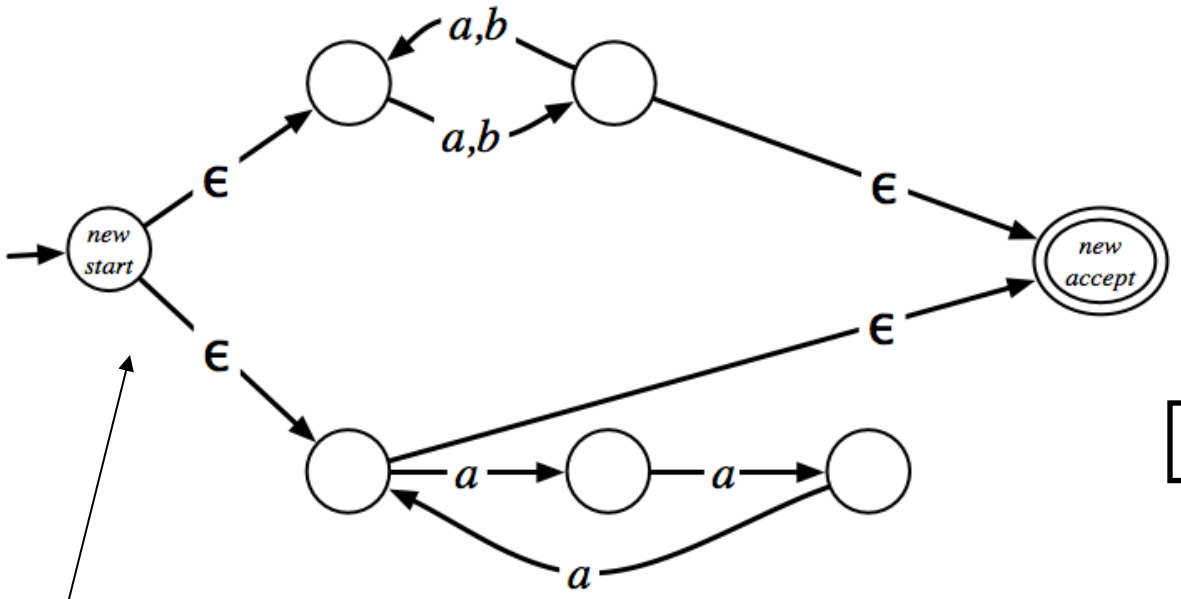

TRUE

FALSE



# NFAs Closed Under ***Union***, Concatenation

- For NFAs $N_1$,$N_2$, there is some NFA $N_3$:
  - $L(N_3) = L(N_1) \cup L(N_2)$




W.l.o.g. only one accept state

Non-deterministically choose which NFA to run

# NFAs Closed Under Union, ***Concatenation***

- For NFAs $N_1$,$N_2$, there is some NFA $N_3$:
  - $L(N_3) = L(N_1) \circ L(N_2)$



# NFAs Closed Under Union, ***Concatenation***

- For NFAs $N_1$,$N_2$, there is some NFA $N_3$:
  - first make $N_1$ and $N_2$ only one accepting state each
  - $L(N_3) = L(N_1) \circ L(N_2)$




$N_1$

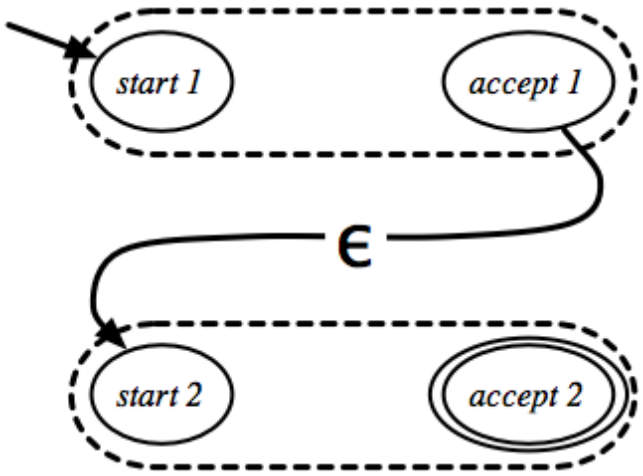



$N_2$



# NFAs Closed Under Union, ***Concatenation***

- For NFAs $N_1$,$N_2$, there is some NFA $N_3$:
  - $L(N_3) = L(N_1) \circ L(N_2)$




$N_3$



# NFAs Closed Under ***Star***

- For any $N_1$ there is some $N_2$ with $L(N_2) = L(N_1)^*$



# NFAs Closed Under ***Star***

- For any $N_1$ there is some $N_2$ with $L(N_2) = L(N_1)^*$

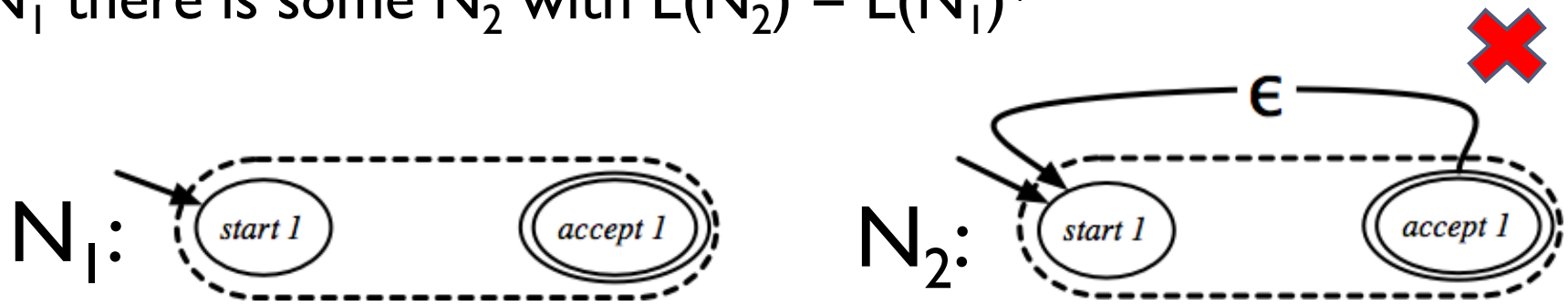



Correct?



# NFAs Closed Under ***Star***

- For any $N_1$ there is some $N_2$ with $L(N_2) = L(N_1)^*$

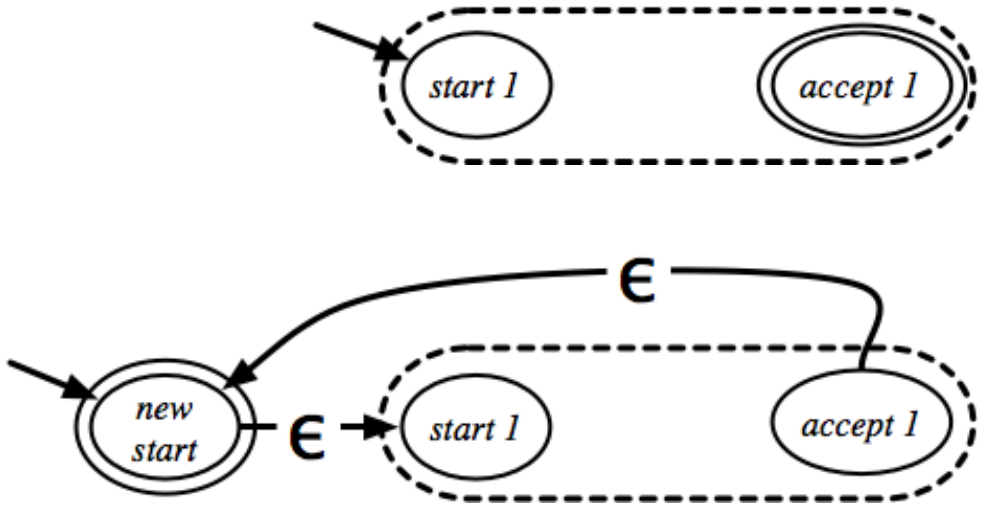



This construction makes $L(N_2) = L(N_1) \circ L(N_1)^*$

*(the concatenation of <u>one</u> or more members of $L(N_1)$.)*

# NFAs Closed Under ***Star***

- For any NFA $N_1$ there is some NFA $N_2$ such that $L(N_2) = L(N_1)^*$




$N_1$

$N_2$



Recall:

- ***Regular Language*** : one accepted by a DFA
- ***Regular Operations*** : union, concat., star

What we know so far:

- The set of languages recognized by NFAs is closed under all three Regular Operations
- Every DFA is also an NFA
- ***Theorem:*** Every language recognized by an NFA is recognized by an equivalent DFA

→ Regular Languages are closed under Regular Operations



# Equivalence of NFAs, DFAs, and Regular Expressions

# “Regular”

- ***Regular Language***
  - All languages accepted by NFAs/DFAs
- ***Regular Operations*** (on sets of strings)
  - Union, Concatenation, Star
- ***Regular Expressions*** (aka *REGEXP*)
  - Defined recursively by operations Union, Concatenation, Star



# REGEXPs

- A ***regular expression*** is a string over:
  - symbols in **Σ** (the alphabet) as well as ∅,ε,∪,∘,*,(,)
- *Regular expressions* represent *languages*
  - *a* for some *a* in the alphabet **Σ**
  - ε
  - ∅
  - $(R_1 \cup R_2)$
  - $(R_1 \circ R_2)$
  - $(R_1^*)$

*where $R_1$, $R_2$ are regular expressions*



# REGEXPs

- A ***regular expression*** is a string over:
  - symbols in **Σ** (the alphabet) as well as ∅,ε,∪,∘,*,(,)
- *Regular expressions* represent *languages*
  - *a* short for {a}, the language with one string: a
  - ε short for {ε}, the language with one string: ε
  - ∅
  - $(R_1 \cup R_2)$ normal definition of union, concat., star
  - $(R_1 \circ R_2)$
  - $(R_1^*)$



# REGEXPs

- Simplified notation:
  - don't write ∘ (xy short for x∘y)
  - order of precedence: *, ∘, ∪ (minimizes need for parentheses)

- Example:
  - (01 ∪ 10)*0* short for ((((0∘1) ∪ (1∘0))*)∘((0)*))

- Also a subset $A \subseteq \Sigma$ can be in the language to denote $\cup_{a \in A}\{a\}$



# Exercises

- Assume $\Sigma = \{0,1\}$
  - $0^*10^*$ = {w | w contains exactly one 1}
  - $\Sigma\Sigma^*$ = $\{w \mid w \neq \varepsilon\}$
  - $(\Sigma\Sigma\Sigma)^*$ = {w | length of w is a multiple of 3}
  - $0^*\emptyset 1^*$ = $\emptyset$



# Example: Password Policies

- *All passwords must begin & end in a letter and contain at least one number and one punct. mark*

- Alphabet: $\Sigma = L \cup N \cup P$ (sets of letters, numbers, puncts.)



# Example: Password Policies

- *All passwords must begin & end in a letter and contain at least one number and one punct. mark*

- Alphabet: $\Sigma = L \cup N \cup P$ (sets of letters, numbers, puncts.)
  - $L\Sigma^*(N\Sigma^*P \cup P\Sigma^*N)\Sigma^*L$



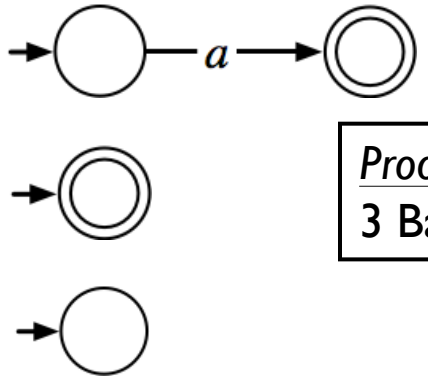
# REGEXPs

- *Every language represented by REGEXPs is recognized by an NFA*
  - $a \in \Sigma$
  - $\varepsilon$
  - $\emptyset$
  - $(R_1 \cup R_2)$
  - $(R_1 \circ R_2)$
  - $(R_1^*)$



# REGEXPs

- *Every language represented by REGEXPs is recognized by an NFA.*
  - $a \in \Sigma$
  - $\varepsilon$
  - $\emptyset$
  - $(R_1 \cup R_2)$
  - $(R_1 \circ R_2)$
  - $(R_1^*)$




*Proof by structural induction*
3 Base Cases

*Inductive Step.*
Assume, inductively, that we have NFAs recognizing langs. of $R_1$ and $R_2$

Use closure of NFAs over $\cup, \circ, ^*$



# REGEXPs equivalent to NFAs

(1) The language of any REGEXP is recognized by an NFA

- *Proof: construct the NFA directly from the REGEXP using union, concat, star operations*

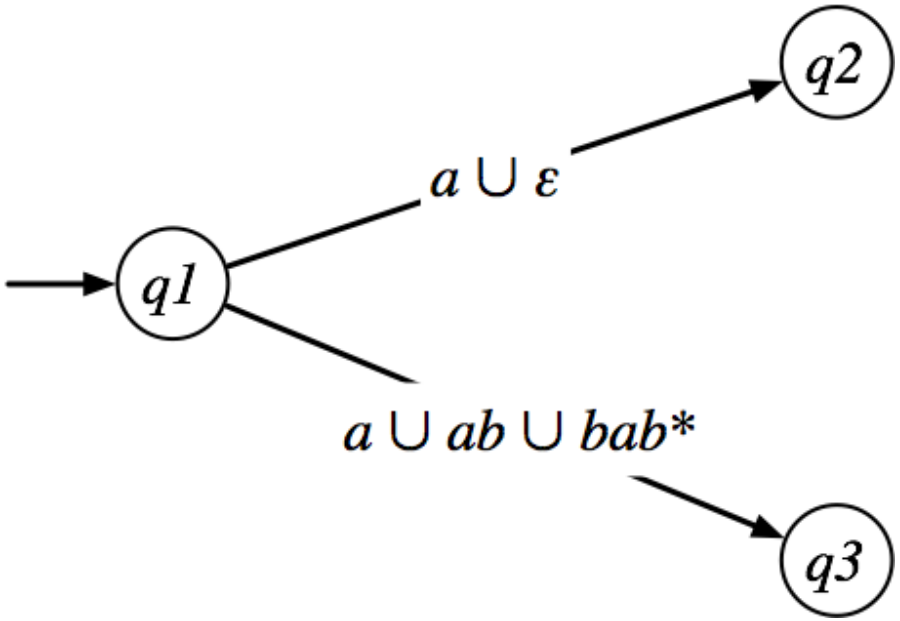
(2) The language of any NFA can be represented by a REGEXP

- *Proof: not at all obvious!*



# ***Generalized*** NFAs

- ***Transitions are REGEXPs***, not just symbols from $\Sigma_\varepsilon$
- Trans. from $q_i$ to $q_j$ after reading any string in $\delta(q_i, q_j)$
- ***Several options from state $q_1$***
  - Read ***a***, go to $q_2$
  - Read nothing, go to $q_2$
  - Read ***a***, go to $q_3$
  - Read ***ab***, go to $q_3$
  - Read ***ba***, go to $q_3$
  - Read ***bab***, go to $q_3$
  - Read ***babb***, go to $q_3$
  - Read ***babbb***, go to $q_3$
  - …






# ***Generalized*** NFAs

- A GNFA is a 5-tuple $(Q,\Sigma,\delta,q_{start},q_{accept})$
  - $Q$ = set of states — same as NFA
  - $\Sigma$ = alphabet — same as NFA
  - $q_{start}$ = start state — same as NFA
  - $q_{accept}$ = accept state
  - $\delta : (Q \setminus \{q_{accept}\}) \times (Q \setminus \{q_{start}\}) \rightarrow \text{REGEXP}$

***How is this definition different from DFA or NFA?***



# ***Generalized*** NFAs

- A GNFA is a 5-tuple $(Q,\Sigma,\delta,q_{start},q_{accept})$
  - Q = set of states — same as NFA
  - $\Sigma$ = alphabet — same as NFA
  - $q_{start}$ = start state — same as NFA
  - $q_{accept}$ = accept state
  - $\delta : (Q \setminus \{q_{accept}\}) \times (Q \setminus \{q_{start}\}) \rightarrow$ REGEXP

    ***3 differences:***

    - *$\delta$ takes two states and gives a REGEXP*
    - *No transitions to $q_{start}$*
    - *No transitions from $q_{accept}$*



- ***Claim:*** Every language recognized by an NFA can be represented as a REGEXP
- ***Proof Strategy:***

  (1) Every NFA is equivalent to a GNFA

  *trivial*

  (2) Every GNFA with k>2 states is equivalent to a GNFA with k-1 states

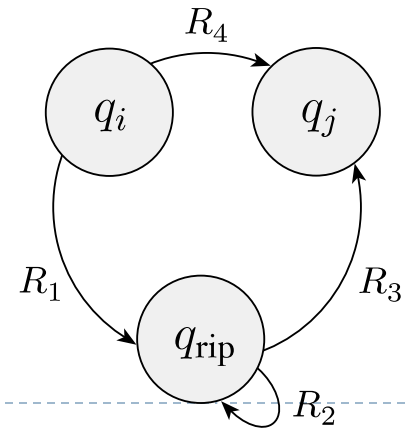
  (3) Every GNFA with 2 states is equivalent to a REGEXP

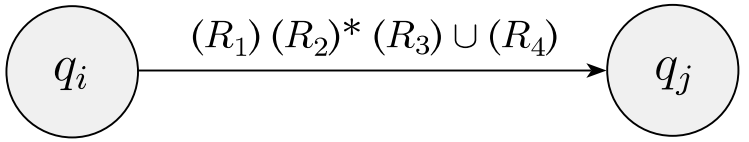


# Reducing the size of GNFAs

- If we want to get rid of a state $q_{rip}$:
- Ways to get from any pair of $q_i$ to $q_j$
  - ***Directly***: read a string in $R_4$, go to $q_j$
  - ***Via $q_{rip}$***: read a string in $R_1$, zero or more strings in $R_2$, a string in $R_3$, then go to $q_j$

- Replace $R_4$ by $R_4 \cup R_1(R_2)^*R_3$, splice out $q_{rip}$
- Delete all non-start/accept states one by one

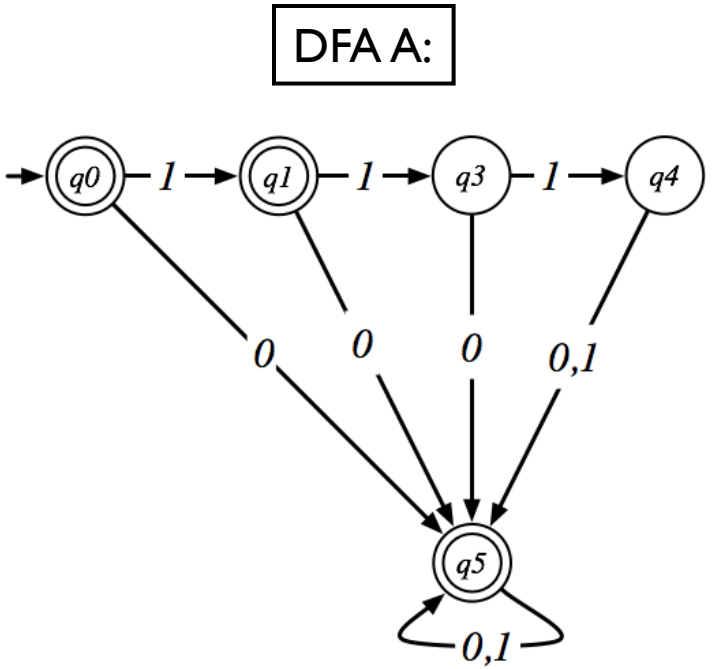





# Example

- L(A) = {w | w is not 11 or 111}

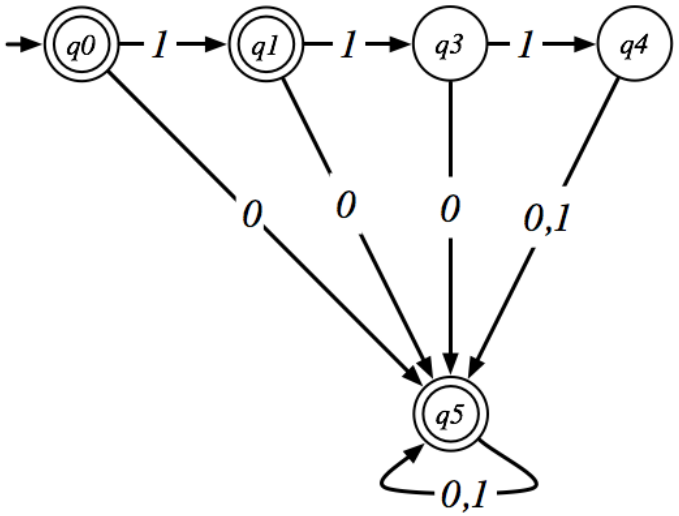





# Example

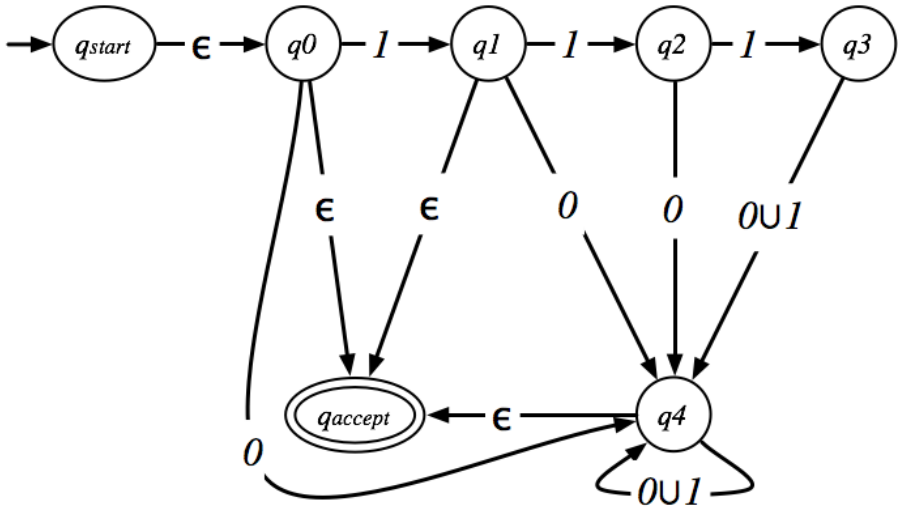
- L(A) = {w | w is not 11 or 111}

DFA A:




first make it a GNFA w/ one start, one accept state

What is $\delta(q_1,q_0)$?



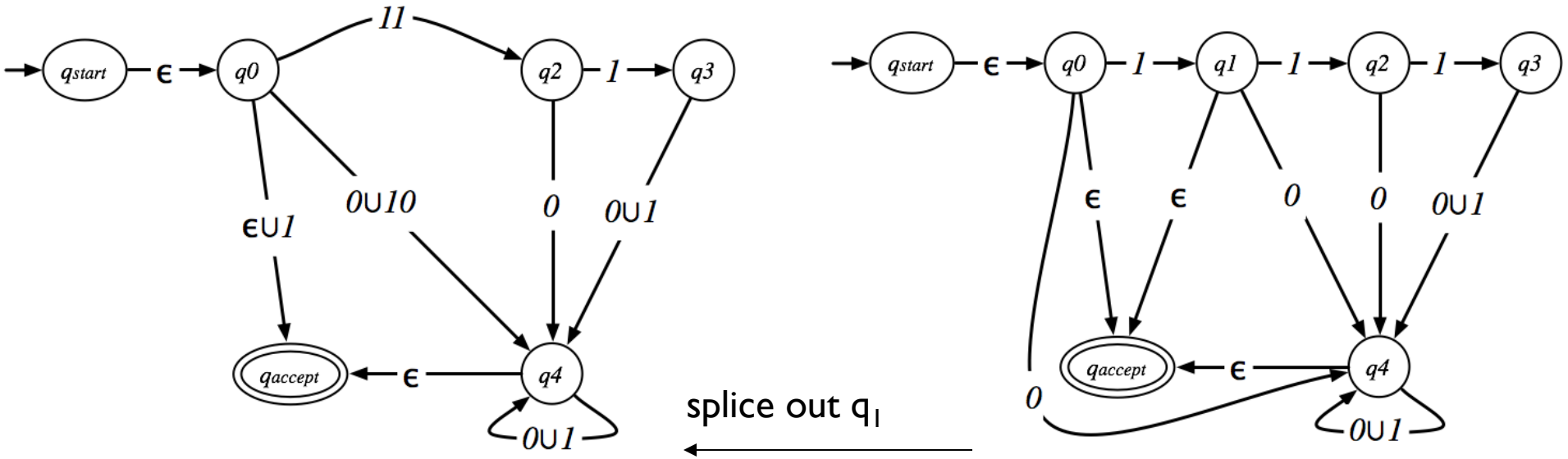

*No transitions to $q_{start}$*

*No transitions from $q_{accept}$*

# Example

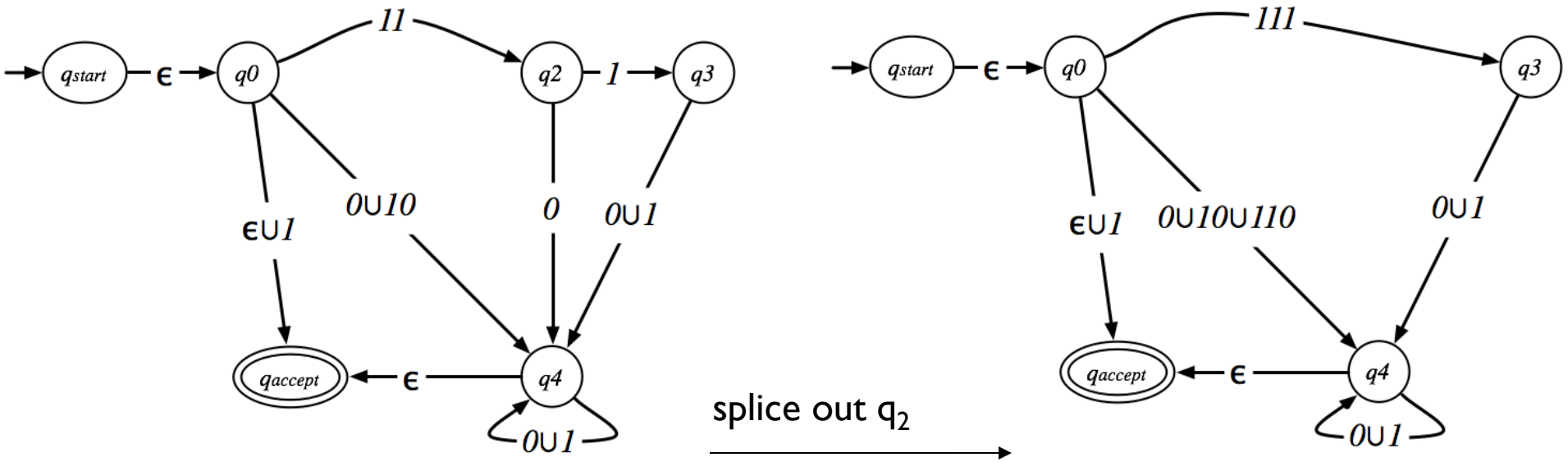
- L(A) = {w | w is not 11 or 111}

All missing transitions are ∅



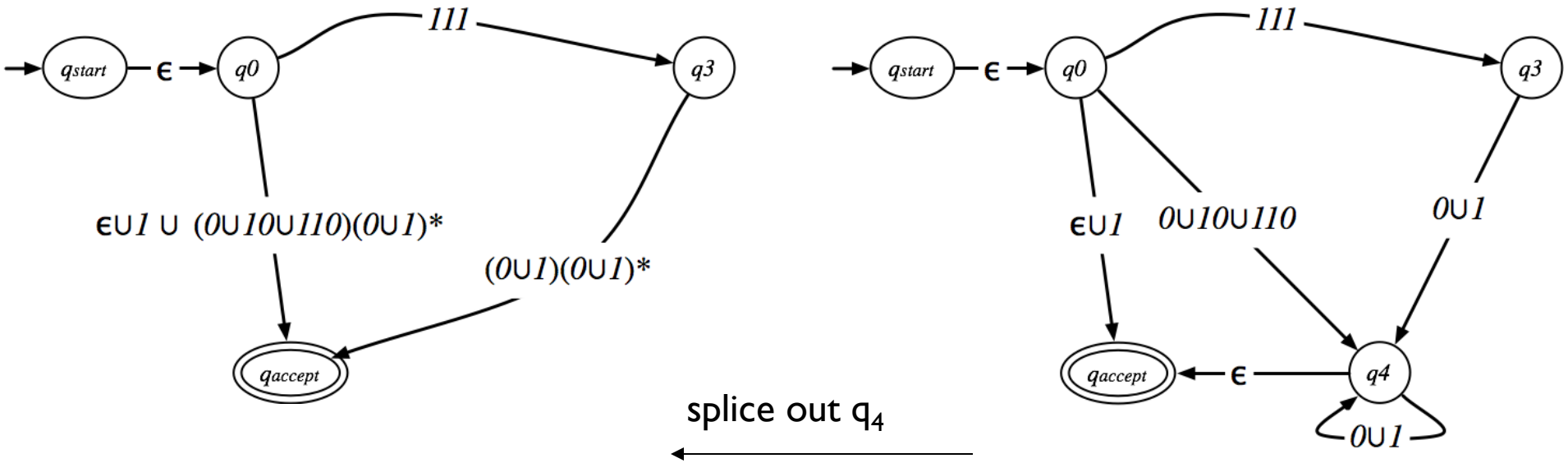
# Example

- L(A) = {w | w is not 11 or 111}



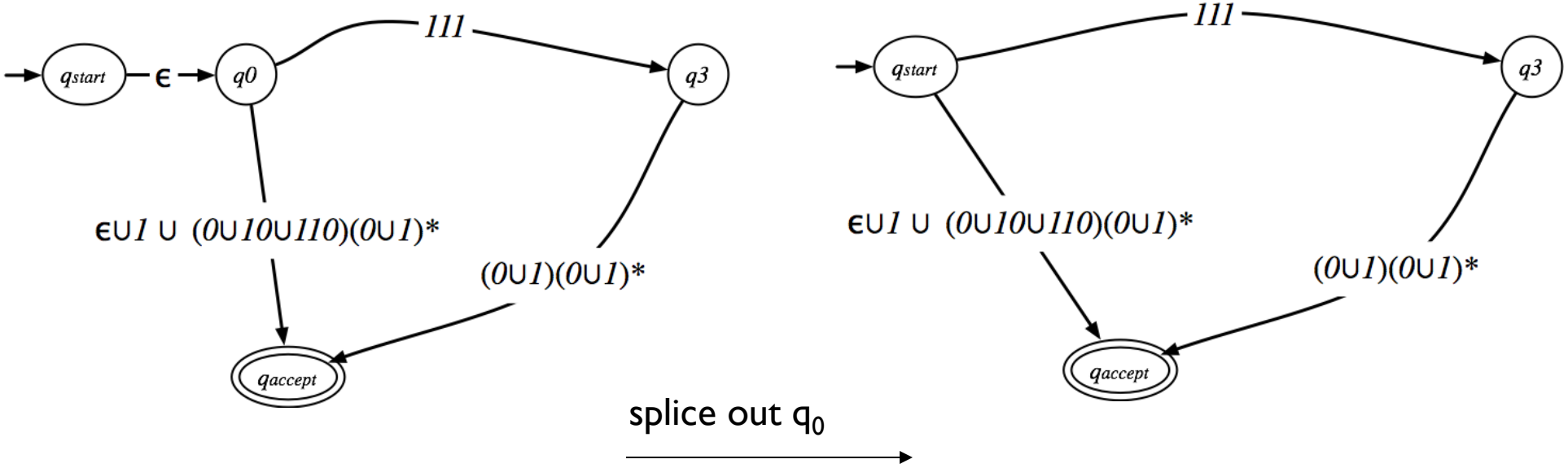
# Example

- L(A) = {w | w is not 11 or 111}



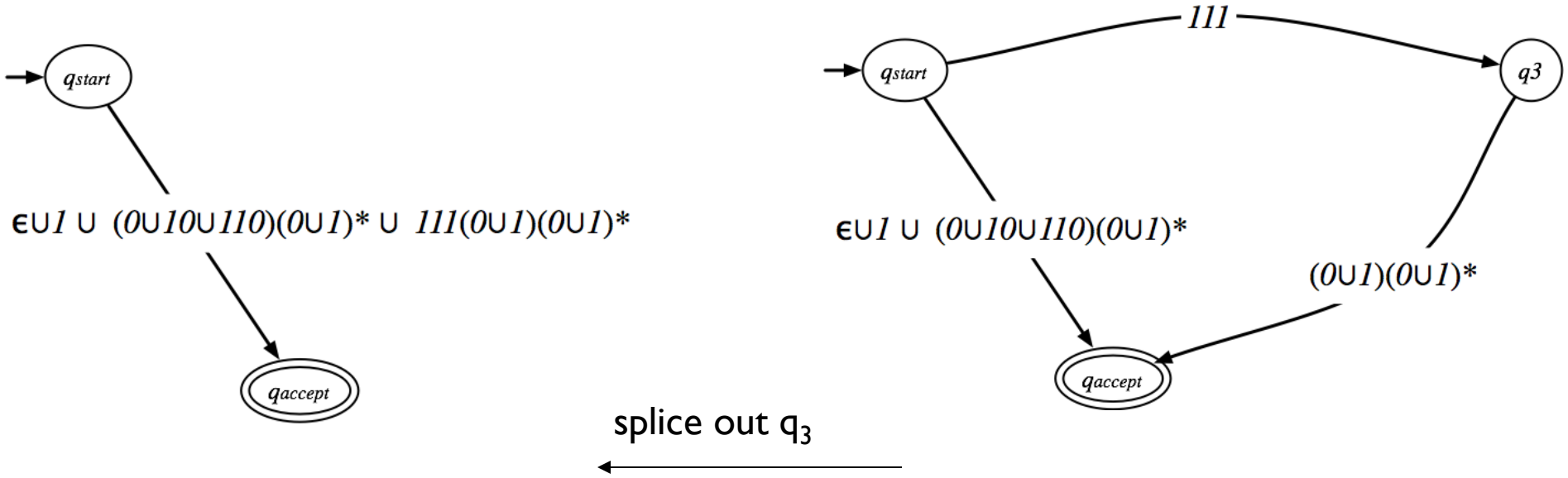
# Example

- L(A) = {w | w is not 11 or 111}



# Example

- L(A) = {w | w is not 11 or 111}



# Equivalence of NFAs, DFAs, and Regular Expressions

- Three equivalence way to define regular languages:
  - recognized by a DFA
  - recognized by an NFA
  - can be expressed by regular expression



# Nonregular Languages

# Nonregular Languages

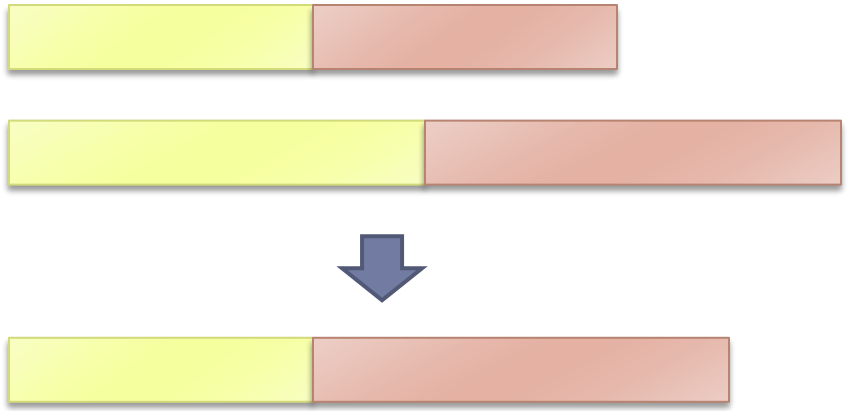
- Is the language $B=\{0^n1^n \mid n\geq 0\}$ regular?



# Nonregular Languages

- Is the language $B=\{0^n 1^n \mid n \geq 0\}$ regular?

- Suppose there is a DFA M with k states recognizing it.
  - Then there must exist $0 \leq i,j \leq k$ such that after reading $0^i$ and $0^j$, M is in the same state. (*Pigeonhole principle*)
  - Then if M accepts $0^i 1^i$, it must accept $0^j 1^i$





# Pumping Lemma

- If A is a regular language, then there is a number p (the pumping length) where, if s is any string in A of length at least p, then s can be divided into s=xyz, satisfying:
  - For each $i \geq 0$, $xy^iz \in A$
  - $|y| > 0$
  - $|xy| \leq p$



# Pumping Lemma

- If A is a regular language, then there is a number p (the pumping length) where, if s is any string in A of length at least p, then s can be divided into s=xyz, satisfying:
  - For each $i \geq 0$, $xy^iz \in A$
  - $|y|>0$
  - $|xy| \leq p$

Proof: Let $M=(Q, \Sigma, \delta, q_{start}, F)$ be a DFA recognizing A and p be the number of states of M
If $|s| \geq p$, let $q_0=q_{start}$, $q_i=\delta(q_{i-1},s[i])$
Then by pigeonhole principle, among $q_0,\ldots, q_p$, there exist $q_i=q_j$ $(i<j)$
So let $x=s[1,\ldots,i]$, $y=s[i+1,\ldots,j]$, $z=s[j+1,\ldots,p]$ *(z can be empty)*
$xy^i$ will also reach $q_i$, so if $xyz \in A$, then $xy^iz \in A$



# Some nonregular languages

- F={ww|w∈{0,1}*}
- D={ $1^{n^2}$ | n≥0}



# Some nonregular languages

- $F=\{ww|w\in\{0,1\}^*\}$
  - Assume there exists a DFA recognizing it, let p be the pumping length
  - Let $s=0^p10^p1$
  - Any $0^{p+i}10^p1$ for $i>0$ are not in F

- $D=\{ 1^{n^2} \mid n\geq 0\}$



# Some nonregular languages

- F={ww|w∈{0,1}*}
  - Assume there exists a DFA recognizing it, let p be the pumping length.
  - Let $s=0^p10^p1$.
  - Any $0^{p+i}10^p1$ for i>0 are not in F.

- D={ $1^{n^2}$ | n≥0}
  - let p be the pumping length
  - then $1^{p^2} \in D$, but $|xy|\leq p$, $0<|y|\leq p$, $|xy^2z|\leq p^2+p$, which cannot be a perfect square



# Another example

- Show that $\{0^i 1^j \mid i>j\}$ is not regular

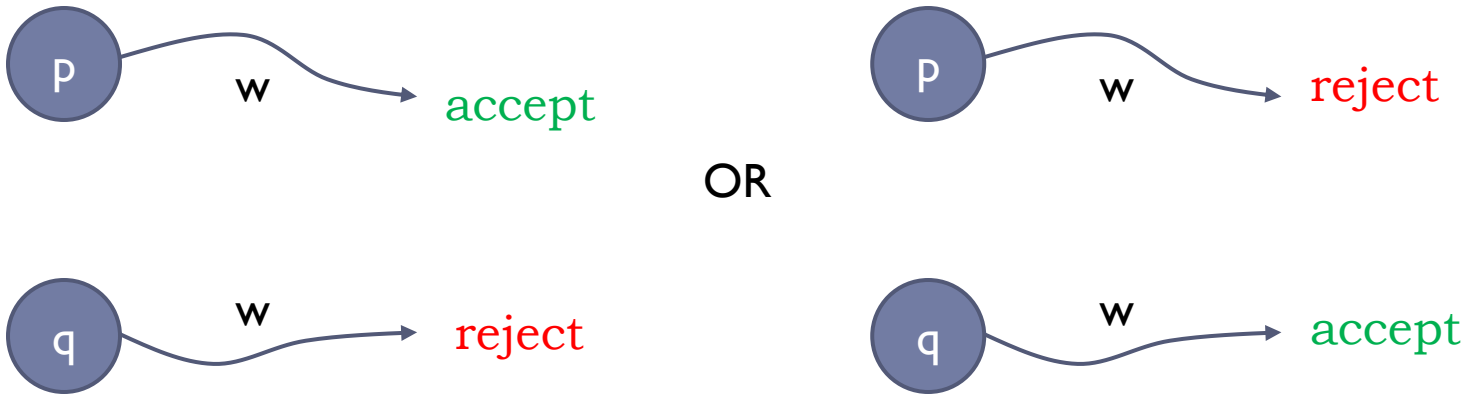


# DFA minimizing algorithm

# Distinguishable states

- For a DFA M, define $M_p$ be the DFA same as M but start at state p in it

- State p is distinguishable from state q if there is a $w \in \Sigma^*$ such that $M_p(w) \neq M_q(w)$






# Distinguishable states

- For a DFA M, define $M_p$ be the DFA same as M but start at state p in it.
- State p is distinguishable from state q if there is a $w \in \Sigma^*$ such that $M_p(w) \neq M_q(w)$
- State p is indistinguishable from state q if for all $w \in \Sigma^*$, $M_p(w) = M_q(w)$
- Pairs of indistinguishable states are redundant



- You can see indistinguishable states (p ~ q) form an equivalent relation



# Algorithm to minimize DFA:

- Delete unreachable states
- Find indistinguishable states

- In a DFA=$(Q,\Sigma,\delta,q_0,F)$
  - For all $p,q \in Q$ such that $p \in F$ and $q \notin F$, mark $(p,q)$
  - If there exist $p,q \in Q$ and $a \in \Sigma$ such that $(\delta(p,a),\delta(q,a))$ is marked, mark $(p,q)$
- Repeat until no pairs can be marked
- Unmarked pairs are indistinguishable states



# Correctness

- The DFA M obtained is the minimal one (# states)
  - Since all states $p_1,\ldots,p_m$ in M are reachable, there exist strings $w_1,\ldots,w_m$ so that after reading $w_i$ we reach $p_i$
- Suppose there is a DFA N with fewer states equivalent to M
  - there exist $w_i$ and $w_j$ so that after reading $w_i$ and $w_j$, M is in states $p_i$ and $p_j$ resp., but N is in the same state q
  - for all string x, $N(w_ix)=N(w_jx)$, this will mean $p_i$ and $p_j$ are indistinguishable
- In fact, any minimal DFA equivalent to M is isomorphic to M

